

More on Context-Free Grammars

Nguyen Phuong Thai

LLOs

- LLO1: Trình bày được các kiến thức cơ bản về phân tích cú pháp bao gồm bài toán và các tiếp cận giải quyết, các mô hình văn phạm CFG và PCFG, phân tích cú pháp theo tiếp cận thống kê và treebank, đánh giá kết quả phân tích cú pháp
- LLO2: Vận dụng được văn phạm CFG/PCFG trong mô tả cú pháp của ngôn ngữ tự nhiên như đọc hiểu dữ liệu treebank, dựng cây cú pháp thủ công, tính xác suất của cây
- LLO3: Vận dụng được các thuật toán: nhị phân hóa văn phạm phi ngữ cảnh (chuyển văn phạm sang dạng chuẩn Chomsky), lập bảng phân tích sử dụng thuật toán CKY

Outline

- Context free grammars (CFGs)
- Building a simple Vietnamese CFG
- Ambiguation
- Chomsky normal form
- Syntactic relations/dependencies
- Special sentence structures
- Passive sentences

Why CFGs are important?

- Representing NL syntax
- Automatic parsing
- Limit representation for analyzing huge number of sentences syntax
- Basic for other formalism such as HPSGs, LFGs, etc.

Formal Definition of CFG

- A grammar G is a quadruple (T , N, S, R), where
  - T : a finite set of terminal symbols or tokens
  - N: a finite set of nonterminal symbols ($T \cap N = \emptyset$)
  - S: a unique start symbol ($S \in N$)
  - R: a finite set of rules or productions of the form (A, α)
  - where:
    - A is a nonterminal, and
    - α is a string of zero or more terminals and nonterminals
- Note: zero means that $\alpha = \varepsilon$ is possible

Backus-Naur Form (BNF)

- A notation for writing a CFG
- To recognise P. Naur's contributions as editor of the ALGOL60 report and J.W. Backus for applying the notation to the first FORTRAN compiler.
- Each production (A, α) is written as:

$$A \rightarrow \alpha$$

  where the arrow $\rightarrow$ means "is defined to be", "can have the form of", "may be replaced with" or "derives"

- Can abbreviate the left to the right

$$\begin{array}{lcl} A & \rightarrow & \alpha_1 \\ A & \rightarrow & \alpha_2 \\ & \vdots & \\ A & \rightarrow & \alpha_n \end{array} \quad \Longrightarrow \quad \begin{array}{lcl} A & \rightarrow & \alpha_1 \\ & | & \alpha_2 \\ & \vdots & \\ & | & \alpha_n \end{array}$$

  where:
  - $\alpha_1, \ldots, \alpha_n$ are the alternatives of A
  - the vertical bar $|$ reads "or else"

BNF (cont)

- Non-terminals: capital letters like A, B, and S
- The start symbol: S
- Strings drawn from $(T \cup N)^*$: lower case Greek letters like α, β, and γ
- Strings of terminals: Lower case Roman letters like u, v, and w

Example

- Write a CFG for generating

  “Bò vàng gặm cỏ non”

  “Trâu ăn cỏ”

- A possible CFG

  S -> NP VP

  NP -> N | N A

  VP -> V NP

  N -> “bò” | “trâu” | “cỏ”

  A -> “vàng” | “non”

  V -> “gặm” | “ăn”

Derivations; Sentential Forms; Sentences; Languages

A grammar derives sentences by

1. beginning with the start symbol, and
2. repeatedly replacing a nonterminal by the right-hand side of a production with that nonterminal on the left-hand side, until there are no more nonterminals to replace.
   - Such a sequence of replacements is called a derivation of the sentence being analysed
   - The strings of terminals and nonterminals appearing in the various derivation steps are called sentential forms
   - A sentence is a sentential form with terminals only
   - The language: the set of all sentences thus derived

The Structure of Parse Trees

- The start symbol is always at the root of the tree.
- Nonterminals are always interior nodes.
- Terminals are always leaves in the tree.
- The sentence being analysed is the the leaves read from left to right.

Constructing a Simple Vietnamese CFG

- Noun phrase
- Verb phrase
- Sentence
- Adverbial

Noun Phrase

- Basic structure of a Vietnamese noun phrase

  <pre-modifier> <head noun> <post-modifier>
- Example: “một mái tóc đẹp”
  - Head noun: “mái”
  - Premodifier: “một”
  - Postmodifier: “tóc”, “đẹp”

Noun Phrase: Premodifier

- Structure:

  <position -2> <position -1>
- Example: « tất cả những chiếc kẹo »
  - <position -2>: “tất cả”
  - <position -1>: “những”
- <position -2>: “tất cả”, “hết thảy”, v.v.
- <position -1>: number, determiner
-

Noun Phrase: Postmodifier

- Much more complex than premodifier
- Postmodifier: noun, adjective phrase, verb phrase, number, pronoun (at the end), prepositional phrase, subordinate clause.
- Example 1: “cái máy tính của cơ quan”
- Example 2: “cái máy tính mà tôi mới mua hôm qua”

Question

- “quả bóng xanh”
- “bản đồ hàng lậu”

Answer

- (NP (Nc quả) (N bóng) (A xanh))
- (NP (N bản đồ) (NP (N hàng) (A lậu)))

Verb Phrase

- Basic structure of a Vietnamese verb phrase
  <pre-modifier> <verb> <post-modifier>
- Example: “đang ăn cơm”

Verb Phrase: Premodifier

- Premodifier: adverb
- Adverb groups:
  - *đã, sẽ, đang, từng, còn, chưa, sắp, v.v.*
  - *không (chẳng, chả), có, chưa*
  - *cũng, vẫn, đều, lại, cứ, chỉ*
  - *rất, hơi, khí, quá*
  - *thường, hay, năng, ít, hiếm, v.v.*

Verb Phrase: Postmodifier

- Postmodifier
  - Complement
  - Adjunct
- Complement
  - Noun phrase: “đá bóng”
  - Verb phrase: “cần viết thư”
  - Prepositional phrase: “chuyển hàng xuống thuyền”
  - Two noun phrases: “tặng bạn quyển sách”
  - Noun phrase and prepositional phrase: “pha cà phê với sữa”
  - Clause: “nói rằng cô ấy đẹp”
  - …

Verb Phrase: Postmodifier

- Adjunct
  - Adverbs: rồi, đã, đi, nào, …
  - Adjective phrases: “chạy rất nhanh”
  - Temporal noun phrases: “đá bóng hôm qua”
  - Prepositional phrases: “đá bóng ở sân Mỹ Đình”
  - or subordinate clauses: “không đi đá bóng được vì trời mưa”

Sentence

- Structure

  <subject> <predicate>
- Subject
  - Noun phrase
  - Verb phrase: “dậy đúng giờ thật khó”
  - Clause: “anh nói thế không đúng”
- Predicate
  - Verb phrase
  - Adjective phrase: “nhà anh ấy xa”
  - Noun phrase: “em bé bảy tuổi”

Ambiguous Grammars

- A grammar is ambiguous if it permits
  - **more than one parse tree for a sentence,**

    or in other words,
  - **more than one leftmost derivation or more than one rightmost derivation for a sentence.**

Coping With Ambiguous Grammars

- Method 1: Rewrite the grammar to make it unambiguous.
- Method 2: Use disambiguating rules to throw away undesirable parse trees, leaving only one tree for each sentence.

Example

S -> NP VP | NP AP

NP -> N | N PP | P

VP -> V NP | V NP PP | V AP | R V NP R

PP -> E NP

AP -> A R | A R AP

P -> chúng tôi

N -> hàng | thuyền | ông | ông già | căng-tin | trường | cửa

V -> chuyển | mở | đi

E -> xuống | của

A -> già | nhanh

R -> đi | quá | mới | lại

Example (cont)

- Analyze the following sentences
  - Chúng tôi chuyển hàng xuống thuyền.
  - Ông già đi nhanh quá.
  - Căn-tin của trường mới mở cửa lại.

Chomsky Normal Form

- A useful form for dealing with context free grammars is the Chomsky normal form.
- This is a particular form of writing a CFG which is useful for understanding CFGs and for proving things about them.
- It also makes the parse tree for derivations using this form of the CFG a binary tree.

CNF (cont)

- So what is Chomsky normal form?
- A CFG is in Chomsky normal form when every rule is of the form A -> B C and A -> a
  - where a is a terminal, and A , B , and C are variables
  - further B and C are not the start variable.
- Additionally we permit the rule S -> ε where S is the start variable, for technical reasons.

CNF (cont)

- The first step is simple! We just add a new start variable S0 and the rule S0 -> S where S is the original start variable.
- By doing this we guarantee that the start variable doesn't occur on the right hand side of a rule.

CNF (cont)

- Next we remove the ε rule. Suppose we are removing the rule A -> ε.
- But now we have to “fix” the rules which have an A on their right-hand side.
  - for each occurrence of A on the right hand side, adding a rule (from the same starting variable) which has the A removed.
  - further if A is the only thing occurring on the right hand side, we replace this A with ε. Of course this latter fact will have created a new ε rule.
- Repeat the above process over and over again until all ε rules have been removed.

CNF (cont)

- For example, suppose our grammar contains the rule A -> ε and the rule B -> uAv where u and v are not both the empty string.
- First we remove A -> ε. Then we add to this rule the rule B -> uv .
  - Make sure that you don't delete the original rule B -> uAv .
- If, on the other hand we had the rules A -> ε and B -> A, then we would remove the A -> ε and replace the rule B -> A with the rule B -> ε. Of course we now have to eliminate this rule via the same procedure.

CNF (cont)

- Next we need to remove the unit rules. If we have the rule A -> B, then whenever the rule B -> u appears, we will add the rule A -> u (unless this rule was already replaced.)
- Again we do this repeatedly until we eliminate all unit rules.

CNF (cont)

- At this point we have converted our CFG to one which
  - has no ε transitions
  - all rules are either of the form variables goes to terminal, or of the form variable goes to string of variables and terminals with two or more symbols.

CNF (cont)

- To convert the remaining rules to proper form, we introduce extra variables. In particular suppose A -> u1u2 … un where n > 2. Then we convert this to a set of rules, A -> u1A1, A1 -> u2A2, …, An-2 -> un-1un.
- Now we need to take care of the rules with two elements on the right hand side.
  - If both of the elements are variables, then we are fine.
  - But if any of them are terminals, we 2 add a new variable and a new rule to take care of these. For example, if we have A -> u1B where u1 is a terminal, then we replace this by A -> U1B and U -> u1.

CNF (cont)

S -> ASB

A -> aAS | a | ε

B -> SbS | A | bb

CNF (cont)

- First we add a new start symbol

  S0 -> S

  S -> ASB

  A -> aAS | a | ε

  B -> SbS | A | bb

CNF (cont)

- Next we need to eliminate the ε rules. Eliminating A -> ε yields

  S0 -> S

  S -> ASB | SB

  A -> aAS | a | aS

  B -> SbS | A | bb | ε

CNF (cont)

- Now we have a new ε rule., B -> ε. Lets remove it

  S0 -> S

  S -> ASB | SB | AS

  A -> aAS | a | aS

  B -> SbS | A | bb

CNF (cont)

- Next we need to remove all unit rules. Lets begin by removing B -> A:

  S0 -> S

  S -> ASB | SB | AS

  A -> aAS | a | aS

  B -> SbS | bb | aAS | a | aS

CNF (cont)

- Further we can eliminate S0 -> S:

  S0 -> ASB | SB | AS

  S -> ASB | SB | AS

  A -> aAS | a | aS

  B -> SbS | bb | aAS | a | aS

CNF (cont)

- Now we need to take care of the rules with more than three symbols. First replace S0 -> ASB by S0 -> AU1 and U1 -> SB:

  S0 -> AU1 | SB | AS

  S -> ASB | SB | AS

  A -> aAS | a | aS

  B -> SbS | bb | aAS | a | aS

  U1 -> SB

CNF (cont)

- Next eliminate S -> ASB in a similar form (technically we could reuse U1 , but lets not):

  S0 -> AU1 | SB | AS

  S -> AU2 | SB | AS

  A -> aAS | a | aS

  B -> SbS | bb | aAS | a | aS

  U1 -> SB

  U2 -> SB

CNF (cont)

- Onward and upward, now fix A -> aAS by introducing A -> aU3 and U3 -> AS:

  S0 -> AU1 | SB | AS

  S -> AU2 | SB | AS

  A -> aU3 | a | aS

  B -> SbS | bb | aAS | a | aS

  U1 -> SB

  U2 -> SB

  U3 -> AS

CNF (cont)

- Finally, fix the two B -> rules:

  S0 -> AU1 | SB | AS

  S -> AU2 | SB | AS

  A -> aU3 | a | aS

  B -> SU4 | bb | aU5 | a | aS

  U1 -> SB

  U2 -> SB

  U3 -> AS

  U4 -> bS

  U5 -> AS

CNF (cont)

- Finally we need to work with the rules which have terminals and variables or two terminals. We need to introduce new variables for these. Let these be V1 -> a and V2 -> b:

  S0 -> AU1 | SB | AS

  S -> AU2 | SB | AS

  A -> V1U3 | a | V1S

  B -> SU4 | V2V2 | V1U5 | a | V1S

  U1 -> SB

  U2 -> SB

  U3 -> AS

  U4 -> V2S

  U5 -> AS

  V1 -> a

  V2 -> b

How to Evaluate Syntactic Trees

- Based on grammatical relations/dependencies
  - Head word-modifier word dependencies
- Example

Special Sentences

- Cháy nhà.
- Nhà cháy.
- Trong túi còn tiền.
- Tiền còn trong túi.
- Trên trời lấp lánh một ngôi sao.
- Một ngôi sao lấp lánh trên trời.